

CIP-A104 — Offensive Security Operations I

Practical Lab 2: File Upload and Command Execution Security Assessment

Student: Loveth Adebayo

Date: 29 August 2026

1. Executive Summary

This report documents a controlled security assessment of file upload and command execution functionality in Damn Vulnerable Web Application (DVWA) and OWASP Mutillidae II. The assessment was performed in an isolated virtual environment using Kali Linux as the attack machine and Ubuntu 22.04 as the target machine.

The investigation demonstrated successful file uploads in both applications and confirmed a command injection vulnerability in Mutillidae at its insecure **Hosed** security level. The command injection was demonstrated using the harmless input `127.0.0.1; whoami`, which returned `www-data`. When Mutillidae was switched to **Security Level 5 (Secure)**, the same payload was blocked.

All testing was conducted within the authorised lab environment using harmless marker files and non-destructive commands.

2. Environment and Scope

Component	Details
Hypervisor	Oracle VirtualBox
Attack Machine	Kali Linux
Kali IP Address	192.168.56.105
Target Machine	Ubuntu 22.04
Ubuntu IP Address	192.168.56.107
Network	Host-Only Network
Applications	DVWA and OWASP Mutillidae II

The assessment was limited to the two intentionally vulnerable applications hosted on the target VM.

3. Investigation Narrative and Results

3.1 DVWA File Upload Assessment

DVWA was initially verified at the Impossible security level (E-1). The file upload functionality was then accessed (E-2), and the harmless marker file lab2-upload-marker.png was selected for upload (E-3).

The application successfully accepted the PNG file and generated a server-side filename, as shown in the successful upload response (E-4). The uploaded file was then successfully retrieved through the browser (E-5) and confirmed through an HTTP request returning 200 OK with a content type of image/png (E-6).

The retrieved file properties were examined in E-7, and its SHA-256 hash was compared with that of the original file in E-8. The hashes differed, indicating that the retrieved file was not byte-for-byte identical to the original local marker file.

A mismatch test was also performed (E-9) to observe DVWA's handling of inconsistent upload characteristics. No harmful server-side code execution was performed during this assessment.

DVWA conclusion: A valid PNG upload was accepted and assigned a generated filename. The file was retrievable from the designated upload location. The assessment did not demonstrate a successful bypass of DVWA's Impossible-level controls.

3.2 Mutillidae File Upload Assessment

The Mutillidae upload page was accessed for testing (E-10). A harmless text marker, lab2-upload-marker.txt, was successfully uploaded, as documented in E-11. The application stored the file under:

```
/tmp/lab2-upload-marker.txt
```

The file was not directly accessible through the web server, as a browser access attempt returned 404 Not Found.

A second harmless marker using the .php extension was then uploaded successfully (E-12). This showed that the tested upload functionality accepted the .php extension without rejecting it at the upload stage. However, no execution of uploaded code was attempted.

Mutillidae upload conclusion: The application accepted both TXT and PHP-extension files in the tested configuration. However, the tested files were stored outside the directly accessible web path, reducing the risk of direct execution through the browser.

3.3 Mutillidae Command Injection Assessment

A normal DNS lookup using 127.0.0.1 was first performed to establish the application's expected behaviour (E-13).

The following harmless input was then tested:

127.0.0.1; whoami

The application returned:

www-data

as shown in E-14. This confirmed that the user input was interpreted as executable command structure, allowing the additional `whoami` command to run.

The security level was then changed to **Level 5 (Secure)**. The same input was tested again and was rejected with the message indicating that malicious characters were not allowed (E-15).

Command injection conclusion: Command injection was confirmed in the insecure configuration. The Secure mode successfully blocked the tested injection payload.

3.4 File Hashes Summary

File	SHA-256
lab2-upload-marker.png (original)	4ded5810edc347f756bda4743eedc8d814281ba05262e55713a9ccb177cf5226
lab2-upload-marker.png (retrieved)	b584d71ebf1b94e6a676272fa325851c735639354f500482337d0151dc5c017
lab2-upload-marker.txt	d351bb9cf91b74f3b2bb893b4f4743f900ed930deccbcf2ffc4e9f73ab731452
lab2-upload-marker.html	aee76ebeda877228a62445e61534461b0be2f9fca5b2c709929c042ace060833
lab2-upload-marker.php	5aa3b559616ba61e701f1c6e5c2e6813f30cad9ba54174efe55a24f6a3a46513

4. Findings and Conclusions

Finding 1: Command Injection Vulnerability Confirmed

Evidence: E-13, E-14

Severity: High in a production environment

The input 127.0.0.1; whoami caused the whoami command to execute and returned www-data. This demonstrates that user input was passed into executable command structure without adequate protection.

Finding 2: Mutillidae Accepted a PHP-Extension Upload

Evidence: E-11, E-12

Severity: Medium

The tested configuration accepted both a harmless TXT file and a file using the .php extension. This demonstrates insufficient file-extension restriction during the upload process.

The risk was partly reduced because the tested upload location was outside the web-accessible path.

Finding 3: Secure Mode Prevented the Tested Command Injection

Evidence: E-15

When Mutillidae was configured to Security Level 5, the same command injection payload was rejected. This demonstrates the effectiveness of stronger input validation against the tested attack.

Finding 4: DVWA Processed a Valid Upload at Impossible Level

Evidence: E-1 to E-9

DVWA accepted a valid PNG file, generated a server-side filename, and allowed retrieval from its upload directory. The original and retrieved files had different SHA-256 hashes, which indicates that the retrieved file was not identical to the original. However, the assessment did not demonstrate a successful security bypass or code execution.

5. Security Recommendations

File Upload Security

1. Validate file extensions using a strict server-side allowlist.

2. Verify file signatures and content rather than trusting client-supplied MIME types.
3. Generate server-side filenames.
4. Store uploads outside the web root.
5. Prevent script execution within upload directories.

Command Injection Prevention

1. Avoid passing user-controlled input directly to system commands.
2. Use safe application APIs instead of shell commands where possible.
3. Apply strict allowlist validation.
4. Use parameterised arguments rather than shell command construction.
5. Run web services with least-privilege accounts.
6. Log and monitor suspicious command input.

6. Overall Conclusion

The assessment successfully evaluated file upload and command execution behaviour in the authorised laboratory environment.

The primary security issue identified was a **command injection vulnerability in Mutillidae's insecure configuration**. The vulnerability was demonstrated using a harmless `whoami` command and was subsequently blocked when the application was switched to Secure mode.

The file upload assessment showed different behaviours between the two applications. DVWA successfully handled a valid PNG upload while operating at the Impossible security level. Mutillidae accepted both a TXT marker and a PHP-extension marker, although the tested uploaded files were stored outside the directly web-accessible path.

The assessment demonstrates the importance of server-side file validation, secure file storage, avoiding direct shell execution, and implementing defence-in-depth controls.

All tests were performed using harmless files and non-destructive commands within the authorised lab environment.

7. Cleanup and Restoration

Following the completion of testing, both vulnerable applications were reset to their baseline states to ensure the environment was restored for future use.

7.1 DVWA Database Reset

The DVWA database was reset using the built-in setup functionality. The reset operation completed successfully, as documented in E-16.

7.2 Mutillidae Database Reset

The Mutillidae database was reset using the Reset DB function available from the application's homepage. The reset operation completed successfully, as documented in E-17.

7.3 Verification Activities

- Clean snapshot verified as available within the virtualisation environment
 - Both applications confirmed to be non-accessible from the host machine
 - All discovery and testing tools were stopped
 - Proxy sessions were saved for potential review
-

Appendix: Evidence Index

Evidence	Filename	Description
E-1	E-1_DVWA_security_level.jpg	DVWA Security Level
E-2	E-2_DVWA_file_upload_Interface.jpg	DVWA File Upload Interface
E-3	E-3_Selected_upload_file.jpg	Selected Upload File
E-4	E-4_Normal_upload_response.jpg	Normal Upload Response
E-5	E-5_DVWA_browser_retrieval.jpg	DVWA Browser Retrieval
E-6	E-6_DVWA_http_retrieval.jpg	DVWA HTTP Retrieval
E-7	E-7_DVWA_Retrieved_properties.jpg	DVWA Retrieved Properties
E-8	E-8_DVWA_integrity_comparision.jpg	DVWA Integrity Comparison
E-9	E-9_DVWA_mismatch_test.jpg	DVWA Mismatch Test
E-10	E-10_Mutillidae_upload_page.jpg	Mutillidae Upload Page
E-11	E-11_Mutillidae_txt_upload_success.png	Mutillidae TXT Upload Success
E-12	E-12_mutillidae_php_upload.png	Mutillidae PHP Upload
E-13	E-13_mutillidae_dns_baseline.png	Mutillidae DNS Baseline
E-14	E-14_mutillidae_command_injection.png	Mutillidae Command Injection
E-15	E-15_mutillidae_secure_mode.png	Mutillidae Secure Mode
E-16	E-16_DVWA_Database_Reset.png	DVWA Database Reset

Evidence Filename		Description
E-17	E-17_Mutillidae_Database_Reset.png	Mutillidae Database Reset